



Chapter 17: Transactions

Database Principle and Design



Outline

- Transactions
- ACID Properties
- Transaction State
- Schedules
- Serializability
- Recoverable Schedules



17.1 Transaction Concepts

- A **transaction** in the database system, usually initiated by a user program, is a series of **database** operations that form a single logical unit of work.
- Two main database operations:
 - **read(X)**, which transfers the data item X from the database to a variable, also called X.
 - **write(X)**, which transfers the value in the variable X to the data item X in the database.
- In addition to reading and writing, each transaction must end with either a **commit** (i.e., complete successfully) or **abort** (i.e., undo all the write operations carried out thus far) operation.
 - By default, a transaction executes **commit** operation as its last step
- Example : transfer \$50 from account A to account B:

```
read(A)
A := A - 50
write(A)
read(B)
B := B + 50
write(B)
commit
```

Note: Although transactions may involve some non-database operations, transaction management does not consider these operations.



ACID Properties

There are four important properties of transactions that a DBMS must ensure to maintain data in the face of concurrent database access and system failures:

- **Atomicity**. Either all database **modifications** in a transaction are carried out or none are.
 - In example 17.3, without the atomicity property, if the fund transfer transaction fails after step 3, money will be “lost” leading to an inconsistent database state.
- **Consistency**. Each transaction, run by itself with no concurrent execution of other transactions, must preserve the consistency of the database.
 - In example 17.3, The sum of A and B is unchanged by the execution of the transaction.
 - Within a transaction, the database may be inconsistent temporarily.
 - Ensuring consistency for an individual transaction is the responsibility of the application programmer who codes the transaction.
 - DBMS must ensure this property when executing transactions.

```
read(A)
A := A - 50
write(A)
read(B)
B := B + 50
write(B)
commit
```

Example 17.3



ACID Properties (Cont.)

- **Isolation.** Concurrent transactions cannot interfere with each other, that is, intermediate transaction results must be hidden from other concurrently executed transactions
 - Without the isolation property, when several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state. Example:

T1
read(A)
 $A := A - 50$
write(A)

read(B)
 $B := B + 50$
write(B)
commit

T2

read(A), **read**(B), **print**(A+B), **commit**

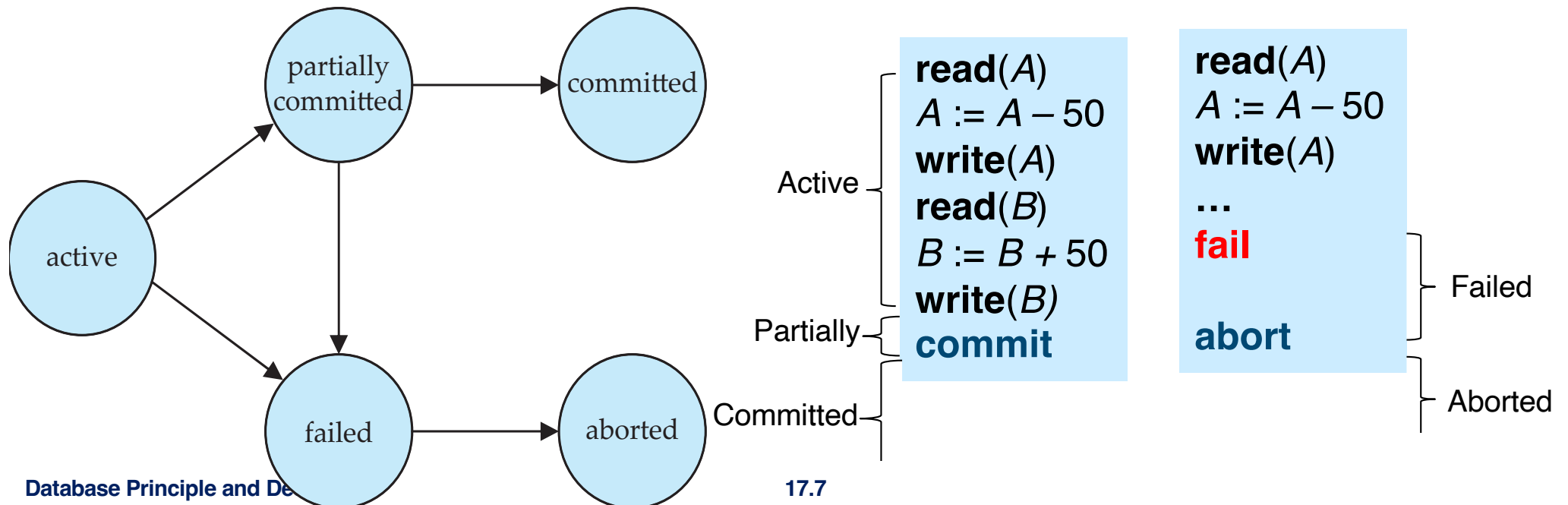
Reading Uncommitted Data
(Dirty Read)

- **Durability.** If a transaction has been successfully committed, its effects should be permanent.



17.2 Transaction State

- **Active** – the transaction stays in this state while it is executing
- **Partially committed** – after the final operation has been executed. (Before **commit** operation is completed)
- **Committed** – after **commit** operation is completed
- **Failed** -- after the discovery that normal execution can no longer proceed. (Before **abort** operation is completed)
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. (After **abort** operation is completed)
- A transaction is said to have **terminated** if it is in either committed or aborted state.





17.3 Transaction Management

- The purpose of **transaction management** is to ensure database consistency after execution of transactions. There are two main components:
 - **Concurrency control scheme** implements the **isolation** property of transactions to ensure database consistency when multiple transactions are executed concurrently.
 - **Recovery scheme** implements the **atomicity** and **durability** properties of transactions to ensure database **consistency** in case of system failure during the execution of transactions.
- Transaction management allows application developers to focus on the implementation of individual transactions, ignoring the issues of concurrency and failure.



17.4 Schedules

- **Schedule** – a sequences of operations that specify the chronological order in which operations of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all operations of those transactions
 - Must preserve the order in which the operations appear in each individual transaction.



Example

T 1 :
 read (A);
 A := A - 50;
 write (A);
 read (B);
 B := B + 50;
 write (B);
 commit.

T 2 :
 read (A);
 temp := A * 0.1;
 A := A - temp;
 write (A);
 read (B);
 B := B + temp;
 write (B);
 commit.

T_1	T_2
read (A) A := A - 50 write (A)	read (A) <i>temp</i> := A * 0.1 A := A - <i>temp</i> write (A)
read (B) B := B + 50 write (B) commit	read (B) B := B + <i>temp</i> write (B) commit

T_1	T_2
read (A) A := A - 50 write (A) read (B) B := B + 50 write (B) commit	read (A) <i>temp</i> := A * 0.1 A := A - <i>temp</i> write (A) read (B) B := B + <i>temp</i> write (B) commit

T_1	T_2
read (A) A := A - 50	read (A) <i>temp</i> := A * 0.1 A := A - <i>temp</i> write (A) read (B)
write (A) read (B) B := B + 50 write (B) commit	B := B + <i>temp</i> write (B) commit



Serial Schedules

- If the operations of different transactions are not interleaved—that is, transactions are executed from start to finish, one by one—we call the schedule a **serial schedule**.
- **A serial schedule** preserves database consistency.

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



Non-Serial Schedules

- Non-serial schedules may not preserve database consistency
- Examples: ($A = 150, B = 350$)

T_1	T_2
read (A) $A=150$ $A := A - 50$ write (A) $A=100$	read (A) $A=100$ $temp := A * 0.1$ $A := A - temp$ write (A) $A=90$
read (B) $B=350$ $B := B + 50$ write (B) commit $B=400$	read (B) $B=400$ $B := B + temp$ write (B) commit $B=410$

T_1	T_2
read (A) $A=150$ $A := A - 50$ $A=100$	read (A) $A=150$ $temp := A * 0.1$ $A := A - temp$ write (A) $A=135$ read (B) $B=350$
write (A) $A=100$ read (B) $B=350$ $B := B + 50$ write (B) commit $B=400$	$B := B + temp$ $B=350+15=365$ write (B) commit $B=365$



Overwriting Uncommitted Data
(Dirty Write)



17.5 Serializability

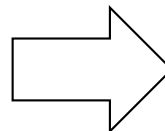
- A schedule is **serializable** if it is equivalent to a serial schedule.
- A serial schedule preserves database consistency, so a **serializable** schedule also does.
- The database system implements **concurrency control** by ensuring that any possibly executed concurrent schedule is **serializable**.



Conflicting Operations

- In a schedule, I and J are two consecutive operations of transactions T_i and T_j
 - I and J **conflict** if they access the same data item Q and at least one of two operations wrote Q .
 - $I = \text{read}(Q), J = \text{read}(Q)$ don't conflict.
 - $I = \text{read}(Q), J = \text{write}(Q)$ conflict.
 - $I = \text{write}(Q), J = \text{read}(Q)$ conflict
 - $I = \text{write}(Q), J = \text{write}(Q)$ conflict
 - If I and J do not conflict, swapping I and J will not change the result of the schedule.

T_1	T_2		T_1	T_2
read(A)			read(A)	
write(A)			write(A)	
	read(A)			read(A)
	write(A)			write(A)
read(B)			read(B)	
write(B)			write(B)	
	read(B)			read(B)
	write(B)			write(B)



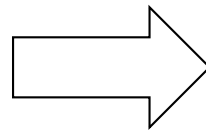


Conflict Serializability

- If a schedule $S1$ can be transformed into a schedule $S2$ by a series of swaps of non-conflicting operations, we say that $S1$ and $S2$ are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule

T_1	T_2
read(A) write(A)	
	read(A) write(A)
read(B) write(B)	
	read(B) write(B)

Schedule 1



T_1	T_2
read(A) write(A) read(B) write(B)	
	read(A) write(A) read(B) write(B)

Schedule 2



Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	write (Q)
write (Q)	

- We are unable to swap operations in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



Other Notions of Serializability

- It is possible that a concurrent schedule is serializable, but not conflict serializable.
- The schedule below produces same outcome as the serial schedule $\langle T_1, T_5 \rangle$, yet is not conflict equivalent to it.

T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

- Therefore, concurrency control based on conflict serializability may reject some serializable schedules



17.6 Recoverable Schedules

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks.
- Example:

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A) commit	read (A) write(A)
The schedule is not recoverable		
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.



Recoverable Schedules (Cont.)

- Database system must ensure that schedules are **recoverable**.
- Recoverable schedule** —if transaction T_j reads a data item previously written by transaction T_i , the **commit** operation of T_i appears before the **commit** operation of T_j .
- Examples:

T_i	T_j
read(A) write(A)	
	read(A) write(A)
read(B) commit	
	 commit



T_i	T_j
read(A) write(A)	
	read(A) write(A) commit
read(B) commit	





Cascadeless Schedules

- Since cascading rollback can lead to the undoing of a significant amount of work. It is desirable to restrict the schedules to those that are **cascadeless**
- **Cascadeless schedule** — cascading rollbacks cannot occur;
 - If transaction T_j reads a data item previously written by transaction T_i , the **commit** operation of T_i appears before the **read** operation of T_j .
 - Example:

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A) commit	read (A) write (A) commit	read (A)

Every cascadeless schedule is also recoverable



End of Chapter 17